

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Колледж программирования и кибербезопасности

УЧЕБНАЯ ДИСЦИПЛИНА/
МЕЖДИСЦИПЛИНАРНЫЙ КУРС
МДК. 02.01 Технология разработки программного обеспечения

ПЦК ***Программирование***

ВИД УЧЕБНОГО
МАТЕРИАЛА ***Лекция - 7***

ПРЕПОДАВАТЕЛЬ ***Понеделко Елена Викторовна***

Лекция – 7

Современные принципы и методы разработки программных приложений

Программы различаются по назначению, выполняемым функциям, формам реализации. Однако можно полагать, что существуют некоторые общие принципы, которые следует использовать при разработке программ.

Частотный принцип

Принцип основан на выделении в алгоритмах и данных особых групп по частоте использования. Для действий, наиболее часто встречающихся при работе программ, создаются условия их быстрого выполнения. К часто используемым данным обеспечивается наиболее быстрый доступ. «Частые» операции стараются делать более короткими. Следует отметить, что лишь не более 5 % операторов программы оказывают ощутимое влияние на скорость выполнения программы. Этот факт позволяет значительную часть операторов программы кодировать без учета скорости вычислений, обращая основное внимание при этом на «красоту» и наглядность текстов.

Принцип модульности

Под модулем в данном контексте понимают функциональный элемент рассматриваемой системы, имеющий оформление, законченное и выполненное в пределах требований системы, и средства сопряжения с подобными элементами или элементами более высокого уровня данной или другой системы. Способы обособления составных частей программ в отдельные модули могут различаться существенно. В значительной степени разделение системы на модули определяется используемым методом проектирования программ.

Принцип функциональной избирательности

Этот принцип является логическим продолжением частотного и модульного принципов и используется при проектировании программ. В программах выделяется некоторая часть важных модулей, которые постоянно должны быть в состоянии готовности для эффективной организации вычислительного процесса. Эту часть в программах называют ядром или монитором. При формировании состава монитора требуется учесть два противоречивых требования. В состав монитора, помимо чисто управляющих модулей, должны войти наиболее часто используемые модули. Количество модулей должно быть таким, чтобы объем памяти, занимаемой монитором, был не слишком большим. Программы, входящие в состав монитора, постоянно хранятся в оперативной памяти. Остальные части программ постоянно хранятся во внешних запоминающих устройствах и загружаются в оперативную память только при необходимости, перекрывая друг друга также при необходимости.

Принцип генерируемости

Основное положение этого принципа определяет такой способ исходного представления программы, который бы позволял осуществлять настройку на конкретную конфигурацию технических средств, круг решаемых проблем, условия работы пользователя.

Принцип функциональной избыточности

Этот принцип учитывает возможность проведения одной и той же работы различными средствами. Особенно важен учет этого принципа при разработке пользовательского интерфейса для выдачи одних и тех же данных разными способами вызова из-за психологических различий в восприятии информации.

Принцип «по умолчанию»

Применяется для облегчения организации связей с системой как на стадии генерации, так и при работе с уже готовыми программами. Принцип основан на хранении в системе некоторых базовых описаний структур, модулей, конфигураций оборудования и данных, определяющих условия работы с программой. Эту информацию программа использует в качестве заданной по умолчанию, если пользователь забудет или сознательно не конкретизирует ее.

Современные методы разработки ПО

Метод нисходящего проектирования (метод пошаговой детализации, метод иерархического проектирования, top-down-подход)

Суть метода заключается в определении спецификаций компонентов системы путем последовательного выделения в ее составе отдельных составляющих и их постепенной детализации до уровня, обеспечивающего однозначное понимание того, что и как необходимо разрабатывать и реализовывать.

Этот метод является незаменимым при разработке сложных по характеру и больших по объему программ, когда к их разработке необходимо привлекать большое число программистов, работающих параллельно. Он позволяет концентрировать внимание разработчиков на наиболее ответственных частях программы, а также облегчает возможность постоянного контроля за ее работоспособностью по мере разработки, отладки и объединения отдельных составляющих программ за счет организации непрерывности этого процесса в течение всей разработки.

Для ускорения разработки программного комплекса часто вместо некоторых программ нижнего уровня, находящихся в процессе разработки, могут применяться специальные "программы-заглушки" Программы-заглушки требуются только на ранних стадиях разработки для того, чтобы не сдерживать общий ход создания программного комплекса. Суть программы-заглушки заключается в том, что при обращении к ней в соответствии с заданным набором исходных тестовых данных она не формирует, а выбирает результат "решения" из заранее подготовленного набора. Благодаря этому обеспечивается возможность имитировать работу на ЭВМ реально создаваемой программы, а следовательно, осуществлять проверку работоспособности программ верхнего уровня еще до того, как будут разработаны и отлажены все составляющие программы нижнего уровня.

Модульное проектирование

Реализация метода *нисходящего проектирования* тесно связана с другим понятием программирования - *модульным проектированием*, так как на практике при декомпозиции

сложной программы возникает вопрос о разумном пределе ее дробления на составные части. Вместе с тем понятие модульности нельзя сводить только к представлению сложных программных комплексов в виде набора отдельных функциональных блоков.

Модуль - это последовательность логически взаимосвязанных фрагментов задачи, оформленных как отдельная часть программы. При этом программные модули должны обладать следующими свойствами:

- на модуль можно ссылаться (т.е. обращаться к нему) по имени, в том числе и из других модулей;
- по завершении работы модуль должен возвращать управление тому модулю, который его вызывал;
- модуль должен иметь один вход и выход;
- модуль должен иметь небольшой размер, обеспечивающий его обозримость.

При разработке сложных программ в них выделяют головной управляющий модуль, подчиненные ему модули, обеспечивающие реализацию отдельных функций управления, функциональную обработку (т.е. непосредственную реализацию основного назначения программного комплекса), а также вспомогательные модули, обеспечивающие сервисное обслуживание пакета (например, сбор и анализ статистики работы программы, обработка различного рода ошибочных ситуаций, обучение и выдача подсказок и т.п.).

Модульный принцип разработки программ обладает следующими преимуществами:

- большую программу могут разрабатывать одновременно несколько исполнителей, и это позволяет сократить сроки ее разработки;
- появляется возможность создавать и многократно использовать в дальнейшем библиотеки наиболее употребимых программ;
- упрощается процедура загрузки больших программ в оперативную память, когда требуется ее сегментация;
- возникает много естественных контрольных точек для наблюдения за осуществлением хода разработки программ, а в последующем для контроля за ходом исполнения программ;
- обеспечивается более эффективное тестирование программ, проще осуществляются проектирование и последующая отладка.

Преимущества модульного принципа построения программ особенно наглядно проявляются на этапе сопровождения и модификации программных продуктов, позволяя значительно сократить затраты сил и средств на реализацию этого этапа.

Структурное программирование

Актуальная для начального периода развития и использования ЭВМ проблема разработки программ, занимающих минимум основной памяти и выполняющихся за кратчайшее время, в последующем в связи резким падением стоимости аппаратной части ЭВМ, значительным возрастанием их быстродействия и объемов памяти сменилась необходимостью разработки и применения принципиально новых методов составления программ. Все это нашло свое воплощение в разработке принципа *структурного программирования.* Одной из целей структурного программирования было стремление облегчить разработку и отладку программных модулей, а главное - их последующее сопровождение и модификацию.

В настоящее время структурное программирование - это целая дисциплина, объединяющая несколько взаимосвязанных способов создания ясных, легких для понимания программ. Эффективность применения современных универсальных языков программирования во многом определяется удобством написания с их помощью структурных программ.

CASE-технологии

За последнее десятилетие в области средств автоматизации программирования сформировалось новое направление под общим названием CASE-технологии (Computer Aided Software Engineering).

CASE-технология представляет собой совокупность средств системного анализа, проектирования, разработки и сопровождения сложных программных систем, поддерживаемых комплексом взаимоувязанных инструментальных средств автоматизации всех этапов разработки программ. Благодаря структурным методам CASE-технология на стадиях анализа и проектирования обеспечивает разработчиков широкими возможностями для различного рода моделирования, а централизованное хранение всей необходимой для проектирования информации и контроль за целостностью данных гарантируют согласованность взаимодействия всех специалистов, занятых в разработке ПО.

Технологии RAD

В начале 80-х годов появилась методология, по которой разработка программы начиналась не после завершения процесса выработки окончательных требований к ней, а как только устанавливались требования на первый, “стартовый” (пилотный) вариант прикладной программы, позволяющий начать содержательную работу по ее реализации на компьютере.

Это дало пользователю возможность, получая уже с первых шагов конкретное представление о характере реализации задачи, уточнять ее постановку. Тем самым облегчался процесс экспериментального поиска нужного решения автоматизации задачи. Благодаря тесному взаимодействию разработчика с заказчиком (пользователем) на самом ответственном этапе создания прикладных программ между ними достигалось быстрое взаимопонимание цели поставленной задачи и возможности ее автоматизации в данных конкретных условиях. Это повышало скорость разработки программ и послужило основанием для названия такой технологии *RAD (Rapid Application Development* - быстрая разработка программ), которая получила широкое распространение.

Data Warehouse

Другое направление разработки прикладных программных средств, олицетворяющее собой современный подход к реализации широкого круга задач для принятия управленческих решений, базируется на концепции создания специального хранилища данных (Data Warehouse). Основное отличие концепции Data Warehouse от традиционного представления баз данных заключается в следующем:

- во-первых, в том, что актуализация данных в Data Warehouse означает не обновление элементов информации, а добавление новых элементов к уже имеющимся (что расширяет возможности проведения различного рода сравнительного анализа);

- во-вторых, в том, что наряду с информацией, непосредственно отражающей состояние системы управления, в Data Warehouse аккумулируются и метаданные.

Метаданные (данные о данных) облегчают возможность визуального представления содержимого Data Warehouse, позволяют, "перемещаясь" по хранилищу, быстро отбирать необходимые данные для последующей обработки.

Основные типы метаданных Data Warehouse отражают:

- структуру и содержимое хранилища;
- соответствие между исходными и выходными данными;
- объемные характеристики данных;
- критерии архивирования;
- отношения между данными;
- информацию по кодированию;
- интервал жизни данных и т.п.

Концепция *Data Warehouse* поддерживается RAD средствами разработки прикладного ПО.

Концепция Data Warehouse обеспечивает возможность разработки программных приложений для поддержки процессов принятия решений с использованием OLAP-систем.

Система OLAP (On-Line Analytical Process) предоставляет возможность разработки информационных систем, ориентированных на уна организацию многомерных баз данных и создание корпоративных сетей, а также обеспечивает поддержку Web-технологий в сетях Internet/Intranet

Успешное применение инструментальных средств OLAP-систем объясняется быстротой разработки приложений, гибкостью и широкими возможностями в области доступа к данным и их преобразования. В настоящее время на рынке ПО предлагается большое число OLAP-стем, разработчиками которых являются различные фирмы, например IBM, Informix, Microsoft, Oracle, Sybase и др.